

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Smart Complaint Management System

Campus Facility Complaint Registration, Routing and Resolution Tracking

SOFTWARE ARCHITECTURE AND DESIGN

Item	Details
Group / Course	Team 6 / UE24CS341A — Software Engineering
Document version	2.0 — simplified planned design
Product scope	SCMS Release 1.0; six planned Agile sprints
Document status	**Prepared for submission**

Team profile

Member / ownership	SRN / PRN
Anushka K Kishore — F1	PES1UG24CS074 / PES1202400461
Atul Suresh — F2	PES1UG24CS098 / PES1202402181
Chaitanya Krishna Makkena — F3	PES1UG24CS122 / PES1202403729
Tanmay Sridhar — F4	PES1UG25CS849 / PES1202503400

1. Introduction / 2. System Architecture

Purpose: describe the SCMS architecture and design as a Django application implementing the SRS. Readers: Team 6 and course reviewers.

2.1 Pattern and rationale

Retain the layered, single-application Django design: templates capture/display data; forms/views validate and authorize requests; functional business logic applies complaint rules; Django ORM/models persist records. This is a modular monolith, not independently deployed services. It supports clear student ownership and unit tests without unnecessary infrastructure.

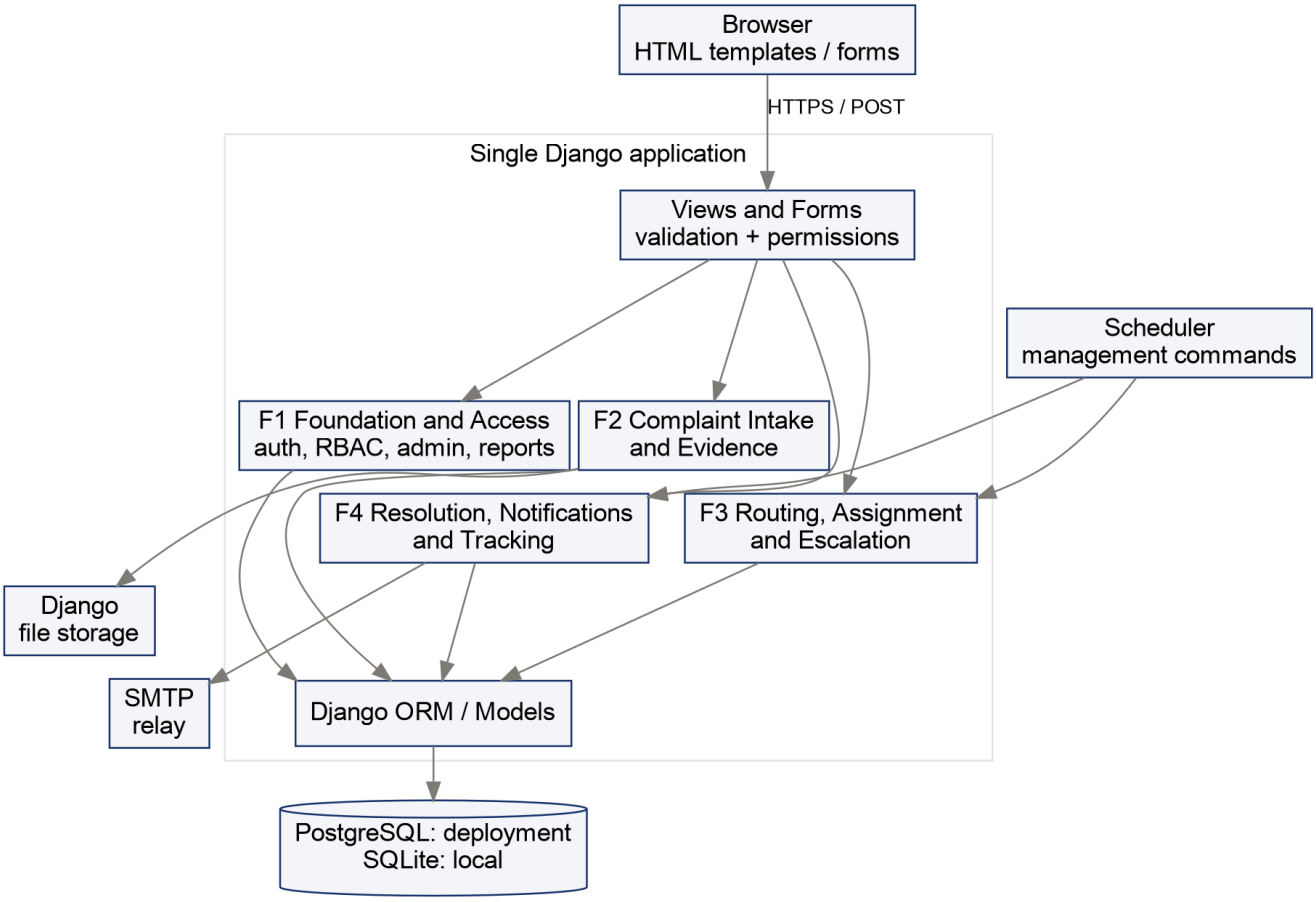


Figure SAD-1. Component architecture. F1–F4 are modules in one application, not microservices.

Deploy on Linux/Docker with PostgreSQL and protected file storage; use SQLite for local development. Scheduled Django commands perform escalation and auto-close, and Django email connects to SMTP. Jenkins and code analysis are development tools, not runtime components.

3. Components / Modules

Component / owner	Responsibility / interfaces
F1 Foundation and Access Anushka K Kishore	Registration, login/logout, password/lockout/session rules, RBAC, departments/staff membership, administrator screens/reports/CSV.
F2 Complaint Intake and Evidence Atul Suresh	Complaint form/validation, unique ID, evidence upload/viewing, anonymous submission/reference tracking and specified lookup throttling.
F3 Routing, Assignment and Escalation Chaitanya Krishna Makkena	Categories/SLA, rules, automatic routing, queues, resolver assignment/reassignment, due dates and scheduled escalation.
F4 Resolution, Notifications and Tracking Tanmay Sridhar	State/history, progress/resolution, reject/reopen/auto-close, in-app/email notices, feedback/rating, Complainant/Head dashboards.

3.1 Internal component contracts

Boundary	Input → output / failure
F1 authentication/permissions	Credentials/current user → authenticated role or generic refusal; role+object/department → allow/deny.
F2 intake → F3 routing	Validated category/location/priority → selected active department, applied rule/default, derived priority and due time. Intake returns ID/reference only after required work succeeds.
F3 allocation → F4 history	Complaint, current Head, same-department target/reason → assignee update and recorded event; invalid scope/target refused.
F4 lifecycle/feedback	Complaint/action/actor/remarks/evidence → permitted state/history/notices; invalid state/actor/data or competing change refused.
Reports	Authorized scope/date range → counts/durations and matching CSV.

These are planned Python call/data boundaries. No public REST API or native binary interface (ABI) is included in the system scope.

3.2 Shared team work

Anushka: access/security review. Atul: pytest/coverage/fixtures. Chaitanya: Jenkins CI/CD. Tanmay: GitHub Projects/repository/documentation. Every member still owns their full F1–F4 functionality and participates in integration and peer review.

4. Data and class design

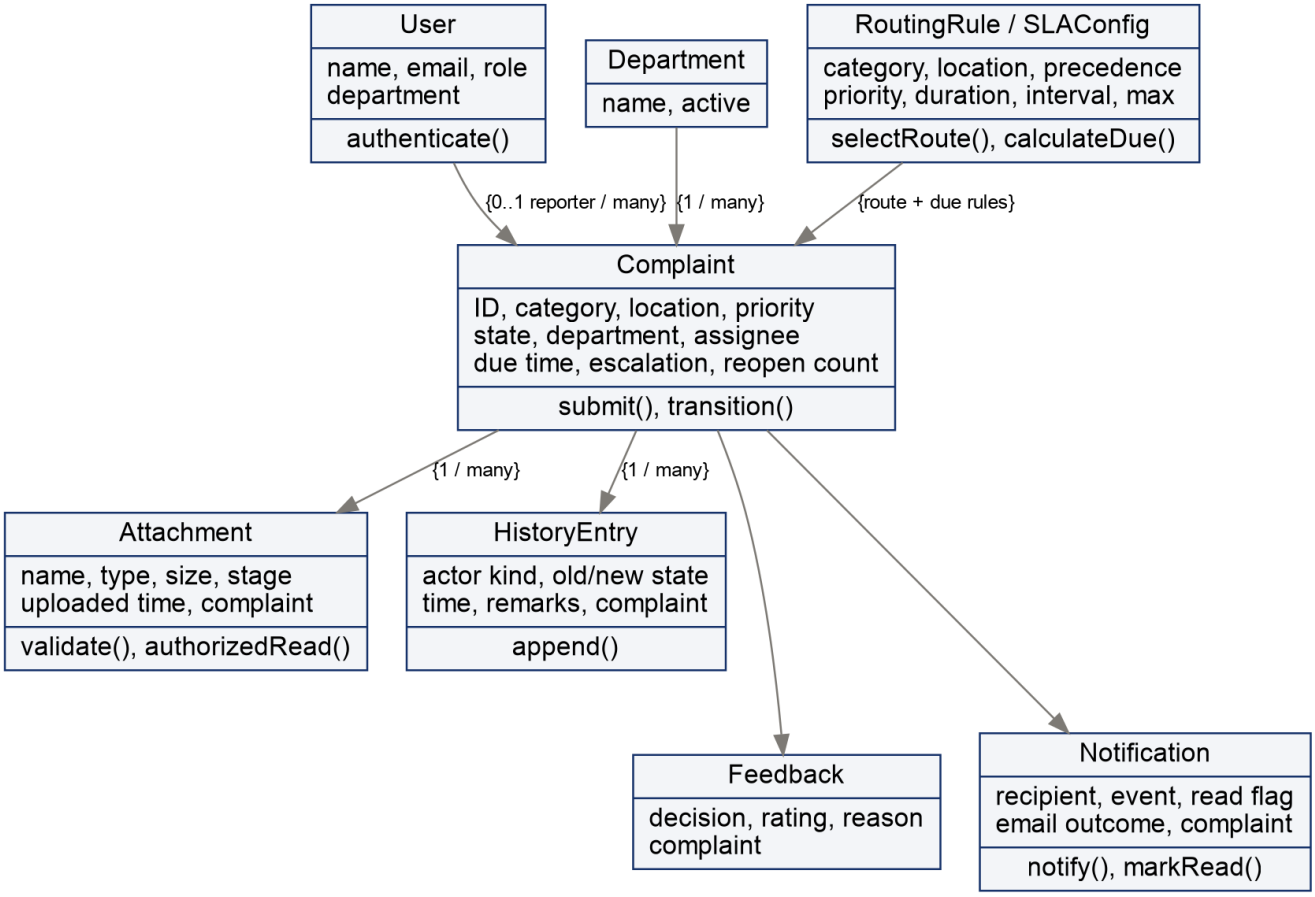


Figure SAD-2. Major domain classes/relationships and illustrative operations.

Entity	Key design rules
User / department	Exactly one user role; Staff/Head department membership; unique email and Django password hash.
Complaint / routing / SLA	Unique public ID; anonymous account null; one department; optional assignee; applied rule or default marker; stored due time, state, escalation/reopen data.
Attachment / history	File bytes via Django storage; metadata in DB; authorized parent access. Append-only history with actor kind/time/remarks.
Feedback / notification	Owner/reference-specific confirmation/rejection/rating; notices for affected users, read status and email outcome/retries.

Use Django ORM, relationships and migrations. Validate field limits;
enforce unique identifiers/email and required references. Configuration changes do not rewrite existing route/due-time decisions. Anonymous history records an anonymous action, not a person/reference/IP.

Exact table names, code files, storage configuration and database migrations will be defined during implementation.

5. Sequence Diagrams — Submit Complaint

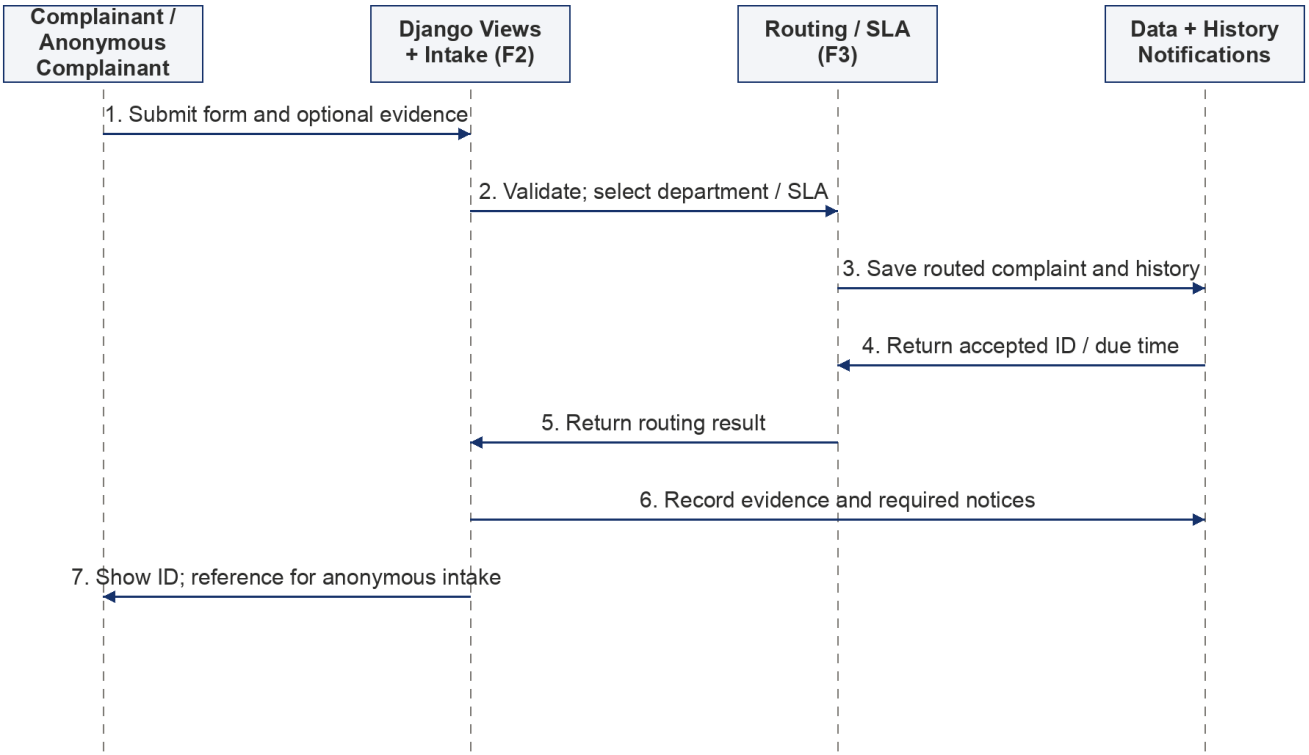


Figure SAD-3. UC-02/UC-03 intake and UC-05 routing. Steps are logical responsibilities within one Django application.

5.1 Flow and failures

Validate the form and evidence, apply active category/location routing with default fallback, compute priority/category due time, and save complaint/evidence/history/notices. Initial SUBMITTED is followed by ASSIGNED for department routing; Head resolver allocation is later. Show the ID only on successful complete intake; anonymous intake additionally shows the one-time reference.

Failed required storage/routing leaves no accepted complaint or evidence; a retry creates no duplicate. Coordinate database changes and clean up failed uploads. Mail failure must not reverse complaint acceptance.

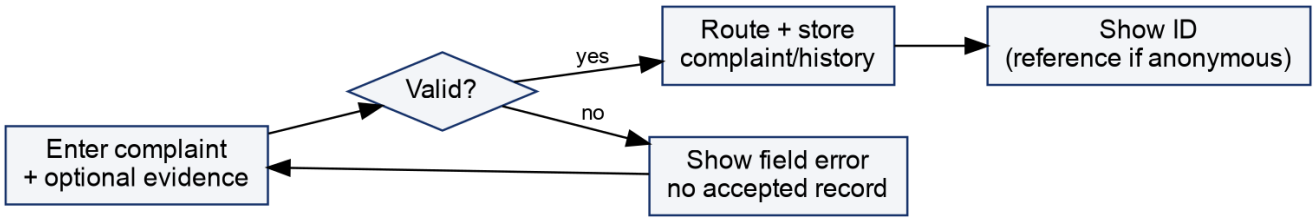


Figure SAD-4. Intake activity: validation error returns to the form without an accepted record.

5. Sequence Diagrams — Resolve Complaint

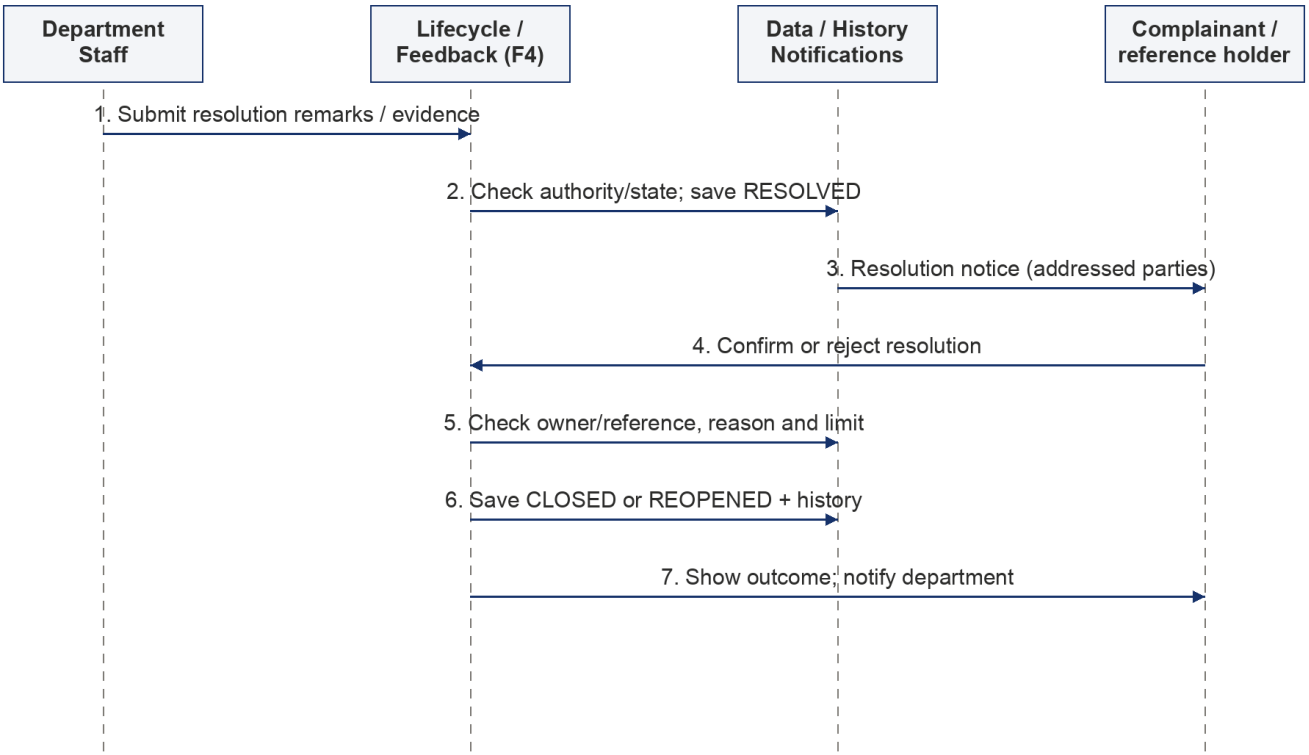


Figure SAD-5. UC-06 resolution and UC-08 owner/reference feedback. Anonymous reporters have no stored email; staff notices still apply.

5.2 Flow and state rules

Assigned Department Staff submits >=20-character remarks and optional valid evidence. Check authority/current state before saving RESOLVED, history and required notices. The originating Complainant or valid anonymous reference holder confirms to CLOSED with optional rating, or rejects with >=20-character reason to REOPENED in the same department.

Reopen count/default maximum 2 and the default 7-day confirmation period are retained. Further rejection raises the matter to the Head without an extra reopen. Scheduled auto-close applies only while still RESOLVED and is recorded unconfirmed. Escalation follows the independent flag/level rules in SRS, not a state change or reassignment.

Reject invalid or competing changes without corrupting state/history. The design requires conflict-safe updates without prescribing a particular locking/version mechanism. Scheduler frequency and increasing email retry delays are implementation settings, not additional features. Reopen due-date reset and exact expiry equality still require confirmation.

6. Web Interface / Route Design

The UI uses server-rendered Django templates/forms, not a public REST API. The route families below are planned contracts. GET displays data; state-changing POST validates CSRF and permission. Form responses are HTML acknowledgements, redirects or useful field errors, not API tokens.

Planned route / method	Input / access	Output / error
/accounts/register/ GET, POST	Name/email/password/type/hostel; public	Form or generic acknowledgement; invalid field/password error.
/accounts/login/ GET, POST; /logout/ POST	Credentials / current session	Role landing or logout; generic failure/lockout.
/complaints/new/ GET, POST	Intake fields/files; Complainant	ID acknowledgement or validation/dependency failure.
/anonymous/new/ GET, POST	Same intake; no identity/login	ID + one-time reference; no partial record on failure.
/anonymous/track/ GET, POST	Reference; public, throttled	Restricted status/history or identical no-match.
/complaints/<id>/ GET; evidence GET	Owner or authorized department reader	Detail/history/protected file; generic denied/not-found.

6.1 Form and error conventions

Inputs must not allow client-supplied role, actor, department assignment or complaint state to override server decisions. Preserve safe fields on validation failure, but not credentials or file bytes. Denied state change/file read reveals no extra complaint data. No stack traces, file paths or SQL details are displayed.

Django forms/views constitute the external interface, with Django ORM calls as internal data access.

6. Web actions and UX (continued)

Planned route / method	Input / authority	Outcome
/complaints/<id>/assign/ or reassign/ POST	Head of department; same-department Staff; reassignment reason	Assignee/history changed, state unchanged; scope/target error.
/complaints/<id>/progress/ or resolve/ POST	Assigned Staff; valid state; resolution remarks/files	Allowed progress/RESOLVED + history/notices; invalid/competing action refused.
Authorized reject action POST	Authority only as SRS state model; complaint/action	REJECTED where permitted; reason recorded with history. Actual route pending.
/complaints/<id>/feedback/ POST; /anonymous/feedback/ POST	Originator or valid anonymous reference; decision/reason/rating	CLOSED/REOPENED or validation/reopen-limit refusal.
/admin/master-data/* POST	Administrator; validated configuration	Create/edit/deactivate/audit; protect references/last Admin.
/reports/ GET; /reports/export/ GET	Administrator global / Head department; date range	Matching HTML/CSV, authorized scope.
Notification read action POST	Current recipient; viewed notice	Read flag/unread count updated; concrete route pending.

6.2 UX design

Use the existing screen inventory: access, submission/acknowledgement, own-complaints/detail, anonymous lookup, staff queue/actions, Head dashboard, administration and reports. Responsive from 320 px; name/role/unread count/logout in authenticated header; text statuses and copyable complaint ID; errors next to fields; confirmation for irreversible actions; paginated/filterable lists.

Registration and submission should take no more than four primary screens. Show emergency-channel notice on both intake paths. Anonymous tracking never becomes full record or evidence access.

6.3 Unresolved definitions

Confirm report date inclusivity and resolution-duration formula, reopen due-date policy, 5 MB byte convention, exact auto-close equality boundary, real category/SLA values and route/module names.

7. Security Architecture / 8. Error Handling

Objective / threat	Control and validation
SO-1 — unauthorized disclosure	Current role/department/ownership and protected files; restricted anonymous view and no identity/reference linkage; HTTPS and safe logs. TC-02/04/05/13; VM-02.
SO-2 — tampering / misuse	Django validation/escaping/ORM, CSRF POST, authorized state model, history/configuration audit; invalid action leaves data unchanged. TC-03/08/12/13.
SO-3 — interrupted operation	No lost/duplicate/partial intake; persistent records/files, bounded escalation and email retries; run outcomes. TC-03/07/11; VM-01.
Spoofing / brute-force attempts	Hash passwords, generic responses, lockout/session expiry, reference throttling. TC-01/04; VM-02.

Security is enforced through application checks. No separate production monitoring stack, distributed queue or advanced infrastructure is specified.

8.1 Errors, logs and operational handling

Invalid input gives field guidance. Unauthorized action/file access is refused. Required intake storage/routing failure yields no accepted record; email failure preserves action/in-app notice and retries up to three times with increasing delay. Reject conflicting state changes. Record security events, append-only history and scheduled-run start/completion/outcome without credentials or anonymous identifying linkage.

8.2 Course delivery workflow

GitHub issues/stories → member work → reviewed PR → Jenkins build/tests/coverage/static/security checks → Docker test deployment. Retain commit, review, pipeline and test records.

9. Requirement Traceability and Validation Update

Component	SRS groups / use cases	Planned checks
F1 Foundation and Access	AUTH/ACC, ADM-01/02/04, RPT; UC-01/09/10	TC-01/02/12/14; VM-02/04
F2 Complaint Intake and Evidence	CMP/ANON; UC-02/03/04; file controls	TC-03/04/05/13; VM-01
F3 Routing, Assignment and Escalation	ROUTE/ASSIGN/SLA, ADM-03; UC-05/07/09	TC-06/07/12; VM-01/05
F4 Resolution, Notifications and Tracking	TRK/LIFE/FDBK/NOT, RPT dashboards; UC-04/06/08/10	TC-08/09/10/11/14; VM-01/02
Cross-cutting	Security, NFRs, business/domain rules	TC-13/15 and VM-01–05; SRS Appendix C.

9.1 Design-to-validation update

Validate module behaviour with pytest; exercise Django form/ORM integration; walk complete use cases through the browser. Test object/file permissions at the server, allowed/denied transitions, routing precedence/default, due/escalation boundaries, anonymous restrictions, mail failure and matching reports. Reliability checks include failure/retry/restart; no internal production mechanism is necessary to explain these outcomes.

9.2 Implementation risks

Risk / gap	Action
Code/build mapping	Map logical modules now; replace Pending implementation with actual files after coding.
Master-data/metric policies unresolved	Confirm rather than invent values/formulas or a reopen SLA reset.
Mail / scheduler availability	Use controlled test backend/clock, then authorized environment checks.
Design not yet implemented	Verify routes, class relationships and controls against actual code; retain **SRS requirements.**

All 57 requirement and rule IDs are mapped in SRS Appendix C and the RTM. Update code mappings and validation results during implementation and test execution.